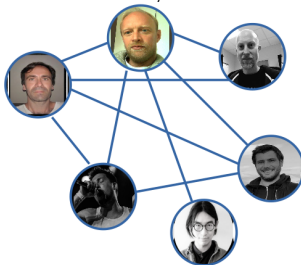




Machine Learning on Graphs

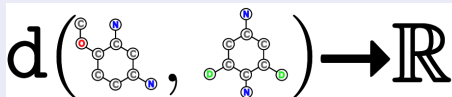
Master 2 Data Science 2025-2026

Stéphane Nicolas / Pierre Héroux / Sébastien Adam / PhD students



Graph-Edit distance

Reminder : our objective for the two first lectures



Our tool of interest : the Graph Edit Distance

$$GED(G_1, G_2) = \min_{\{e_1, \dots, e_k\} \in \Gamma(G_1, G_2)} \sum_{i=1}^k c(e_i)$$

- $c(e_i)$ denotes the "strength" of node edit operation (including the cost of implied edges edit operations)
- GED provides an "explainable" metric
- NB : The minimal cost edit path is not necessarily unique

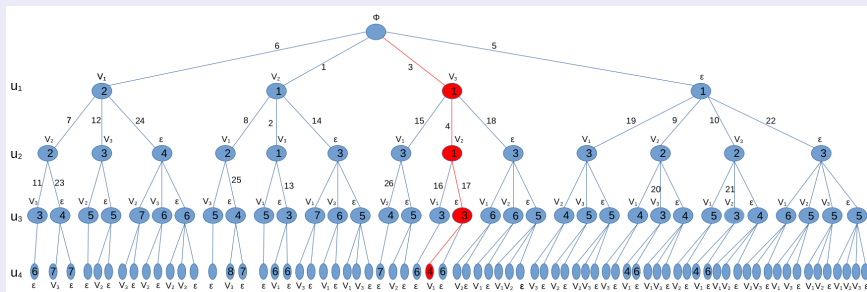
Outline of the lecture

- 1 Lecturer presentation
- 2 Metrics on graphs
- 3 Exact Graph Matching based similarities
- 4 Graph Edit Distance
 - Error-Tolerant graph Matching
 - Definitions and notations
 - Algorithms for exact GED computation
 - Tree-search algorithms
 - Mathematical programming approaches
 - Algorithms for approximate GED computation
 - LSAP approaches
 - QAP approaches
- 5 The Hungarian Method
- 6 Extra-slides : BP-GED refinements

Exact GED algorithms (6)

Previous Lecture

Tree search potentially combined with heuristics (A^*)



Exact GED algorithms (6)

Other framework for GED computing : Binary Linear Programming

Binary Linear Programming (BLP) is a restriction of Integer Linear Programming (ILP) with binary variables. Its general form is :

$$\min_x c^T x \quad (1a)$$

$$\text{subject to } Ax \leq b \quad (1b)$$

$$x \in \{0, 1\}^n \quad (1c)$$

- $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^m$ are the (input) data of the problem.
- A feasible solution is a vector x of n binary variables (1c) which respects linear inequality constraints (1b).
- The optimal solutions are the ones that minimize the objective function (1a) which is a linear combination of variables of x weighted by the components of the vector c .

Exact GED algorithms (6)

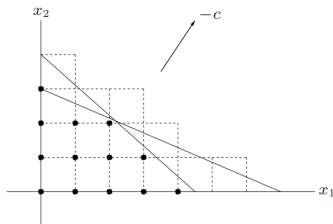
Example of ILP (variables are not binary but integers)

Example

$$\begin{array}{ll}\text{minimize} & -2x_1 - 3x_2 \\ \text{subject to} & (x_1, x_2) \in \mathcal{P}\end{array}$$

where

$$\mathcal{P} = \{x \in \mathbf{Z}_+^2 \mid \frac{2}{9}x_1 + \frac{1}{4}x_2 \leq 1, \quad \frac{1}{7}x_1 + \frac{1}{3}x_2 \leq 1\}$$



optimal point: (2, 2)

Exact GED algorithms (7)

Advantage of this framework (for this problem and many others)

Some dedicated ("blackbox") solvers are available for efficiently solving ILP formulations :

- GLPK
- GUROBI
- CPLEX

Principles used by the solver (remember the RO lecture) :

- "Branch and Bound"
- "Branch and Cut"

Core idea : continuous relaxations as Lower Bounds (e.g. using simplex)

Since we have solvers, our job when using ILP is "only" to write the mathematical formulation of the problem, we feed them with c , A and b , then give the solution

Exact GED algorithms (7)

BLP formulation of GED : which variables ?

Exact GED algorithms (7)

BLP formulation of GED : which variables ?

- $\forall (i, k) \in V_1 \times V_2, x_{i,k} = \begin{cases} 1 & \text{if } i \text{ is substituted with } k, \\ 0 & \text{otherwise.} \end{cases}$
- $\forall (ij, kl) \in E_1 \times E_2, y_{ij,kl} = \begin{cases} 1 & \text{if } ij \text{ is substituted with } kl, \\ 0 & \text{otherwise.} \end{cases}$
- $\forall i \in V_1, u_i = \begin{cases} 1 & \text{if } i \text{ is deleted from } G_1 \\ 0 & \text{otherwise.} \end{cases}$
- $\forall ij \in E_1, e_{ij} = \begin{cases} 1 & \text{if } ij \text{ is deleted from } G_1 \\ 0 & \text{otherwise.} \end{cases}$
- $\forall k \in V_2, v_k = \begin{cases} 1 & \text{if } k \text{ is inserted in } G_1 \\ 0 & \text{otherwise.} \end{cases}$
- $\forall kl \in E_2, f_{kl} = \begin{cases} 1 & \text{if } kl \text{ is inserted in } G_1 \\ 0 & \text{otherwise.} \end{cases}$

Exact GED algorithms (8)

BLP formulation of GED : which objective function ?

Let's suppose the costs given by the problem definition :

$$c(i \rightarrow k), c(ij \rightarrow kl), c(i \rightarrow \epsilon), c(\epsilon \rightarrow k), c(ij \rightarrow \epsilon), c(\epsilon \rightarrow kl)$$

Exact GED algorithms (8)

BLP formulation of GED : which objective function ?

Let's suppose the costs given by the problem definition :

$$c(i \rightarrow k), c(ij \rightarrow kl), c(i \rightarrow \epsilon), c(\epsilon \rightarrow k), c(ij \rightarrow \epsilon), c(\epsilon \rightarrow kl)$$

The objective function is :

$$\begin{aligned} d(G_1, G_2) = \min_{\mathbf{x}, \mathbf{y}, \mathbf{u}, \mathbf{v}, \mathbf{e}, \mathbf{f}} & \left(\sum_{i \in V_1} \sum_{k \in V_2} c(i \rightarrow k) \cdot x_{i,k} + \sum_{ij \in E_1} \sum_{kl \in E_2} c(ij \rightarrow kl) \cdot y_{ij,kl} \right. \\ & + \sum_{i \in V_1} c(i \rightarrow \epsilon) \cdot u_i + \sum_{k \in V_2} c(\epsilon \rightarrow k) \cdot v_k \\ & \left. + \sum_{ij \in E_1} c(ij \rightarrow \epsilon) \cdot e_{ij} + \sum_{kl \in E_2} c(\epsilon \rightarrow kl) \cdot f_{kl} \right) \end{aligned} \quad (2)$$

What is the best solution with these variables and this objective function ?

Exact GED algorithms (8)

BLP formulation of GED : which objective function ?

Let's suppose the costs given by the problem definition :

$$c(i \rightarrow k), c(ij \rightarrow kl), c(i \rightarrow \epsilon), c(\epsilon \rightarrow k), c(ij \rightarrow \epsilon), c(\epsilon \rightarrow kl)$$

The objective function is :

$$\begin{aligned} d(G_1, G_2) = \min_{\mathbf{x}, \mathbf{y}, \mathbf{u}, \mathbf{v}, \mathbf{e}, \mathbf{f}} & \left(\sum_{i \in V_1} \sum_{k \in V_2} c(i \rightarrow k) \cdot x_{i,k} + \sum_{ij \in E_1} \sum_{kl \in E_2} c(ij \rightarrow kl) \cdot y_{ij,kl} \right. \\ & + \sum_{i \in V_1} c(i \rightarrow \epsilon) \cdot u_i + \sum_{k \in V_2} c(\epsilon \rightarrow k) \cdot v_k \\ & \left. + \sum_{ij \in E_1} c(ij \rightarrow \epsilon) \cdot e_{ij} + \sum_{kl \in E_2} c(\epsilon \rightarrow kl) \cdot f_{kl} \right) \end{aligned} \quad (2)$$

What is the best solution with these variables and this objective function ?
Setting all the variables to 0

Exact GED algorithms (9)

BLP formulation of GED : what are the nodes constraints ?

Exact GED algorithms (9)

BLP formulation of GED : what are the nodes constraints ?

$$u_i + \sum_{k \in V_2} x_{i,k} = 1 \quad \forall i \in V_1 \quad (3)$$

$$v_k + \sum_{i \in V_1} x_{i,k} = 1 \quad \forall k \in V_2 \quad (4)$$

The first constraint ensures that each vertex of G_1 is either matched to exactly one vertex of G_2 or deleted from G_1 , the second ensures that each vertex of G_2 is either matched to exactly one vertex of G_1 or inserted in G_1

BLP formulation of GED : the same for edges constraints

$$e_{ij} + \sum_{kl \in E_2} y_{ij,kl} = 1 \quad \forall ij \in E_1 \quad (5)$$

$$f_{kl} + \sum_{ij \in E_1} y_{ij,kl} = 1 \quad \forall kl \in E_2 \quad (6)$$

Exact GED algorithms (10)

BLP formulation of GED : topological constraints

In order to respect the graph topology in the matching, an edge $ij \in E_1$ can be matched to an edge $kl \in E_2$ only if the head vertices $i \in V_1$ and $k \in V_2$, on the one hand, and if the tail vertices $j \in V_1$ and $l \in V_2$, on the other hand, are respectively matched. This constraint can be expressed by

$$y_{ij,kl} \leq x_{i,k} \cdot x_{j,l}$$

Exact GED algorithms (10)

BLP formulation of GED : topological constraints

In order to respect the graph topology in the matching, an edge $ij \in E_1$ can be matched to an edge $kl \in E_2$ only if the head vertices $i \in V_1$ and $k \in V_2$, on the one hand, and if the tail vertices $j \in V_1$ and $l \in V_2$, on the other hand, are respectively matched. This constraint can be expressed by

$$y_{ij,kl} \leq x_{i,k} \cdot x_{j,l}$$

Problem : this constraint is not linear

Exact GED algorithms (10)

BLP formulation of GED : topological constraints

In order to respect the graph topology in the matching, an edge $ij \in E_1$ can be matched to an edge $kl \in E_2$ only if the head vertices $i \in V_1$ and $k \in V_2$, on the one hand, and if the tail vertices $j \in V_1$ and $l \in V_2$, on the other hand, are respectively matched. This constraint can be expressed by

$$y_{ij,kl} \leq x_{i,k} \cdot x_{j,l}$$

Problem : this constraint is not linear

Solution : It can be expressed linearly with the following constraints (McKormick) :

- ij and kl can be matched only if their head vertices are matched :

$$y_{ij,kl} \leq x_{i,k} \quad \forall (ij, kl) \in E_1 \times E_2 \quad (7)$$

- ij and kl can be matched only if their tail vertices are matched :

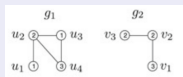
$$y_{ij,kl} \leq x_{j,l} \quad \forall (ij, kl) \in E_1 \times E_2 \quad (8)$$

Exact GED algorithms (11)

F1 formulation

- $|V_1| + |V_2| + |E_1| + |E_2| + |V_1||V_2| + |E_1||E_2|$ variables
- $|V_1| + |V_2| + |E_1| + |E_2| + 2|E_1||E_2|$ constraints

On our toy problem tackled with A^*



33 variables, 29 constraints, my own code (not optimized)

```
Status : Optimal
GED (F1, non dirigé) = 4.0

Substitutions de nœuds (x[i,k] = 1) :
  u2 -> v2 (|2 - 2| = 0)
  u3 -> v3 (|1 - 2| = 1)
  u4 -> v1 (|3 - 3| = 0)

Suppressions de nœuds (u[i] = 1) :
  u1 supprimé (coût 1)

Insertions de nœuds (v[k] = 1) :

Substitutions d'arêtes (y[i,j,k] = 1) :
  ('u2', 'u3') -> ('v2', 'v3') (coût 0)
  ('u2', 'u4') -> ('v1', 'v2') (coût 0)

Suppressions d'arêtes (e[i,j] = 1) :
  ('u1', 'u2') supprimée (coût 1)
  ('u3', 'u4') supprimée (coût 1)

Insertions d'arêtes (f[k,l] = 1) :
```

Exact GED algorithms (11)

F2 formulation

In 2017, a GM student (Pierre Le Bodic), Arnaud Knippel and us proposed a reduced version of this formulation with less variables and constraints :

$$\begin{aligned}
 \min_{\mathbf{x}, \mathbf{y}} & \left(\sum_{i \in V_1} \sum_{k \in V_2} (c_{i,k} - c_{i,\epsilon} - c_{\epsilon,k}) \cdot x_{i,k} \right. \\
 & \left. + \sum_{ij \in E_1} \sum_{kl \in E_2} (c_{ij,kl} - c_{ij,\epsilon} - c_{\epsilon,kl}) \cdot y_{ij,kl} \right) + \gamma \\
 \text{subject to } & \sum_{k \in V_2} x_{i,k} \leq 1 \quad \forall i \in V_1 \\
 & \sum_{i \in V_1} x_{i,k} \leq 1 \quad \forall k \in V_2 \\
 & \sum_{kl \in E_2} y_{ij,kl} \leq x_{i,k} \quad \forall k \in V_2, \forall ij \in E_1 \\
 & \sum_{kl \in E_2} y_{ij,kl} \leq x_{j,l} \quad \forall l \in V_2, \forall ij \in E_1
 \end{aligned}$$

Our paper is available at : <http://www.sciencedirect.com/science/article/pii/S003132031730300X>

Exact GED algorithms (12)

How to solve the BLP

Our implementation of GED using the proposed formulation and some demos on computer vision tasks are available at :

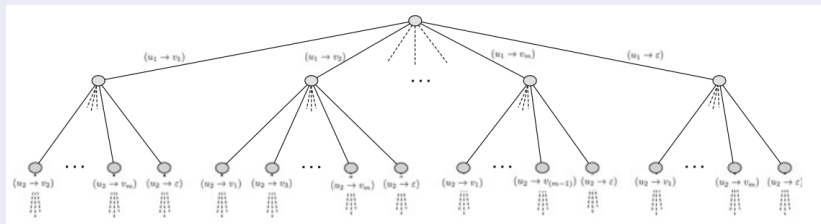
- <https://sites.google.com/site/blpged/>
- <https://github.com/jlerouge/GEMpp>

Other alternatives using BLP

- Before us, Justice and Hero proposed another formulation, dedicated to GED ENA
- We have proposed another formulation called F2 (see <https://github.com/jlerouge/GEMpp>)
- F3 (from Tours) is a refinement for a given class of graphs

Exact GED algorithms (13)

A^* or BLP : same battle : intractable complexity



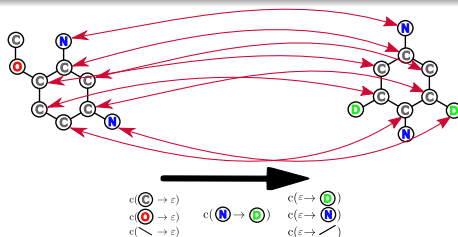
Outline of the lecture

- 1 Lecturer presentation
- 2 Metrics on graphs
- 3 Exact Graph Matching based similarities
- 4 Graph Edit Distance
 - Error-Tolerant graph Matching
 - Definitions and notations
 - Algorithms for exact GED computation
 - Tree-search algorithms
 - Mathematical programming approaches
 - Algorithms for approximate GED computation
 - LSAP approaches
 - QAP approaches
- 5 The Hungarian Method
- 6 Extra-slides : BP-GED refinements

From GED to assignment problems

Result from lecture 1

If conditions on $c()$ are verified, edge operations can be deduced from operation on nodes.



Problem reformulation

GED can be considered as a node assignment problem, a very common Operation Research (OR) problem that consists in finding the optimal permutation $(\varphi_1, \varphi_2 \dots \varphi_N)$

From GED to assignment problems

Assignment problems : usual definition

- Given a set of agents and a set of tasks, with an associated cost for each possible agent-task pair, the goal is to find a matching of maximum cardinality (assigning at most one agent to each task and vice versa) that minimizes the total assignment cost.
- Graph point of view : AP consists of finding, in a weighted bipartite graph, a matching of a given size, which minimizes a cost.
- It is a particular case of Optimal Transport (see Miss Pacheco lecture)
- If the numbers of agents and tasks are equal : balanced assignment
- Otherwise : unbalanced assignment

From GED to assignment problems

Assignment problems : definitions

- If the total cost is equal to the sum of the costs for each agent the problem is called Linear Sum Assignment Problem (LSAP)
- The LSAP can be solved in $\mathcal{O}(n^3)$ time complexity, notably using algorithms such as the Hungarian/Kuh–Munkres algorithm.
- Matrix form : Let C be the cost matrix and X the binary assignment matrix. The problem can be summarized as follows :

$$\min \sum \sum (C \odot X) = \min \quad \text{Tr}(C^T X)$$

With the constraints :

$$X \mathbf{1}_n = \mathbf{1}_n$$

$$X^T \mathbf{1}_n = \mathbf{1}_n$$

$$X_{ij} \in \{0, 1\}$$

From GED to assignment problems

Assignment problems : definitions

- In some assignment problems, the cost can be quadratic (QAP)
- A classical example : the facilities/location problem
 - ▶ a set P of n facilities and a set L of n locations.
 - ▶ a distance $d(l_i, l_j)$ is specified for each pair of locations : the D matrix
 - ▶ a weight or flow $w(p_i, p_j)$ is specified for each pair of facilities (e.g., the amount of supplies transported between the two facilities) : the W matrix.
 - ▶ Objective : assign facilities to locations through a permutation matrix X with the goal of minimizing the sum of the distances multiplied by the corresponding flows : $\sum_{a,b \in P} w(a,b)d(f(a), f(b))$
 - ▶ Matrix form (with the same constraints as LSAP) :

$$\min \quad \text{Tr}(WXD^T X^T)$$

- QAP is NP-complete

From GED to assignment problems

Back to GED

- Problem 1 : GED is an unbalanced AP since the sizes of the graphs can be different

From GED to assignment problems

Back to GED

- Problem 1 : GED is an unbalanced AP since the sizes of the graphs can be different
 - ▶ Solution : extended set of nodes, i.e for two graphs G_1 and G_2 with $|V_1| = n$ and $|V_2| = m$ we use : $V_1^* = V_1 \cup \{\epsilon_1, \epsilon_2, \dots, \epsilon_m\}$ and $V_2^* = V_2 \cup \{\epsilon_1, \epsilon_2, \dots, \epsilon_n\}$: ϵ -assignment
 - ▶ Consequence : the solution is searched in the $(n + m)!$ possible permutations of the integers $1, 2, \dots, (n + m)$

From GED to assignment problems

Back to GED

- Problem 1 : GED is an unbalanced AP since the sizes of the graphs can be different
 - ▶ Solution : extended set of nodes, i.e for two graphs G_1 and G_2 with $|V_1| = n$ and $|V_2| = m$ we use : $V_1^* = V_1 \cup \{\epsilon_1, \epsilon_2, \dots, \epsilon_m\}$ and $V_2^* = V_2 \cup \{\epsilon_1, \epsilon_2, \dots, \epsilon_n\}$: ϵ -assignment
 - ▶ Consequence : the solution is searched in the $(n+m)!$ possible permutations of the integers $1, 2, \dots, (n+m)$

- Problem 2 : the cost to be minimized

$$(\varphi_1, \dots, \varphi_{(n+m)}) = \arg \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \left[\sum_{i=1}^{n+m} c_{i\varphi_i} + \sum_{i=1}^{n+m} \sum_{j=1}^{n+m} c(a_{ij} \rightarrow b_{\varphi_i \varphi_j}) \right]$$

- We have :
 - ▶ a linear term which refers to the sum of node edit costs
 - ▶ a quadratic term which refers to the implied edge edit cost
- B. Gauzere showed that this is an instance of QAP : NP-complete

From GED to assignment problems

From QAP to LSAP

- Problem : solving QAP is NP-complete (it's not a surprise ! we compute the exact GED)
- Naive solution :

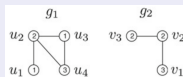
From GED to assignment problems

From QAP to LSAP

- Problem : solving QAP is NP-complete (it's not a surprise ! we compute the exact GED)
- Naive solution : drop the quadratic term and solve a LSAP using an existing polynomial ($O(n^3)$) algorithm for solving LSAP (e.g. Munkres/Hungarian or Volgenant-Jonker)

From GED to assignment problems

GED approximation with LSAP : what are the costs ?



One can easily compute the node assignment cost matrix C :

$$C = \begin{array}{c|cccc|cccc} & v_1 & v_2 & v_3 & & \varepsilon & \varepsilon & \varepsilon & \varepsilon \\ \hline u_1 & 2 & 1 & 1 & & 1 & \infty & \infty & \infty \\ u_2 & 1 & 0 & 0 & & \infty & 1 & \infty & \infty \\ u_3 & 2 & 1 & 1 & & \infty & \infty & 1 & \infty \\ u_4 & 0 & 1 & 1 & & \infty & \infty & \infty & 1 \\ \hline \varepsilon & 1 & \infty & \infty & & & & & \\ \varepsilon & \infty & 1 & \infty & & & & & \\ \varepsilon & \infty & \infty & 1 & & & & & \end{array} \quad \begin{array}{c} \\ \\ \\ \\ \\ 0 \end{array}$$

Once the best assignment has been found, one can compute the true GED. For example, for the non optimal assignment $(2,1,3,7,5,6,4)$, we have :

- $\sum_{i=1}^{n+m} c_{i\varphi_i} = 4$, $\sum_{i=1}^{n+m} \sum_{j=i}^{n+m} c(a_{ij} \rightarrow b_{\varphi_i\varphi_j}) = 4$
- This assignment has a global cost of 8

From GED to assignment problems

From QAP to LSAP

- Problem : edges are not taken into account when solving the assignment problem.
- Intermediate alternative : inject (local) structural constraints in the cost matrix

$$c_{ij}^* = c_{ij} + \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} (c(a_{ik} \rightarrow b_{j\varphi_k}) + c(a_{ki} \rightarrow b_{\varphi_k j})) \quad (\text{directed})$$

$$c_{ij}^* = c_{ij} + \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} c(a_{ik} \rightarrow b_{j\varphi_k}). \quad (\text{undirected})$$

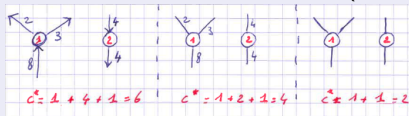
From GED to assignment problems

From QAP to LSAP

- Problem : edges are not taken into account when solving the assignment problem.
- Intermediate alternative : inject (local) structural constraints in the cost matrix

$$c_{ij}^* = c_{ij} + \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} (c(a_{ik} \rightarrow b_{j\varphi_k}) + c(a_{ki} \rightarrow b_{\varphi_k j})) \quad (\text{directed})$$

$$c_{ij}^* = c_{ij} + \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} c(a_{ik} \rightarrow b_{j\varphi_k}). \quad (\text{undirected})$$



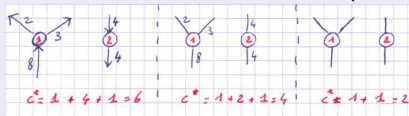
From GED to assignment problems

From QAP to LSAP

- Problem : edges are not taken into account when solving the assignment problem.
- Intermediate alternative : inject (local) structural constraints in the cost matrix

$$c_{ij}^* = c_{ij} + \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} (c(a_{ik} \rightarrow b_{j\varphi_k}) + c(a_{ki} \rightarrow b_{\varphi_k j})) \quad (\text{directed})$$

$$c_{ij}^* = c_{ij} + \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} c(a_{ik} \rightarrow b_{j\varphi_k}). \quad (\text{undirected})$$



- One can see this computing of c_{ij}^* as a new instance of LSAP problem.
- For unlabelled graphs : difference of degrees (in/out for directed)

From GED to assignment problems

New Optimization problem solved using LSAP on C^*

$$(\varphi_1^*, \dots, \varphi_{(n+m)}^*) = \arg \min_{(\varphi_1^*, \dots, \varphi_{(n+m)}^*) \in \mathcal{S}_{(n+m)}} \sum_{i=1}^{n+m} c_{i\varphi_i^*}^*$$

Back to the example with undirected and unlabelled edges

g_1

g_2

$$C = \begin{array}{c|cccc|cccc} & v_1 & v_2 & v_3 & & \varepsilon & \varepsilon & \varepsilon & \varepsilon \\ \hline u_1 & 2 & 1 & 1 & & 1 & \infty & \infty & \infty \\ u_2 & 1 & 0 & 0 & & \infty & 1 & \infty & \infty \\ u_3 & 2 & 1 & 1 & & \infty & \infty & 1 & \infty \\ u_4 & 0 & 1 & 1 & & \infty & \infty & \infty & 1 \\ \hline \varepsilon & 1 & \infty & \infty & & & & & \\ \varepsilon & \infty & 1 & \infty & & & & & \\ \varepsilon & \infty & \infty & 1 & & & & & \end{array} \quad \begin{array}{c|cccc|cccc} & v_1 & v_2 & v_3 & & \varepsilon & \varepsilon & \varepsilon & \varepsilon \\ \hline u_1 & 2 & 2 & 1 & & 2 & \infty & \infty & \infty \\ u_2 & 3 & 1 & 2 & & \infty & 4 & \infty & \infty \\ u_3 & 3 & 1 & 2 & & \infty & \infty & 3 & \infty \\ u_4 & 1 & 1 & 2 & & \infty & \infty & \infty & 3 \\ \hline \varepsilon & 2 & \infty & \infty & & & & & \\ \varepsilon & \infty & 3 & \infty & & & & & \\ \varepsilon & \infty & \infty & 2 & & & & & \end{array}$$

Consider an assignment $(3, 2, 6, 1, 4, 5, 7)$ which is optimal w.r.t the new costs C^* , giving a cost of 6.

One can compute the "real" cost of this assignment : 4

From GED to assignment problems

Deriving Upper and Lower bounds of the GED

- **Upper bound**

- $$d_{\psi}(g_1, g_2) = \sum_{i=1}^{n+m} c_{i\varphi_i^*} + \sum_{i=1}^{n+m} \sum_{j=i}^{n+m} c(a_{ij} \rightarrow b_{\varphi_i^* \varphi_j^*}).$$

- The linear term refers to the node edit operation cost, **computed using** C but with permutations optimized with C^*
- The quadratic term refers to the consistent edge edit operations implied by node operations, also computed using C . It has to be divided by two for undirected graphs.
- On the previous example $(3, 2, 6, 1, 4, 5, 7)$, $d_{\psi}(g_1, g_2) = 4$
- The difference with the true GED is that $\sum_{i=1}^{n+m} \sum_{j=i}^{n+m} c(a_{ij} \rightarrow b_{\varphi_i \varphi_j})$ is not used by LSAP (on the contrary of QAP) . Better solutions may exist !
- The true edit distance can not be greater than $d_{\psi}(g_1, g_2)$

From GED to assignment problems

Deriving Upper and Lower bounds of the GED [Riesen proposal]

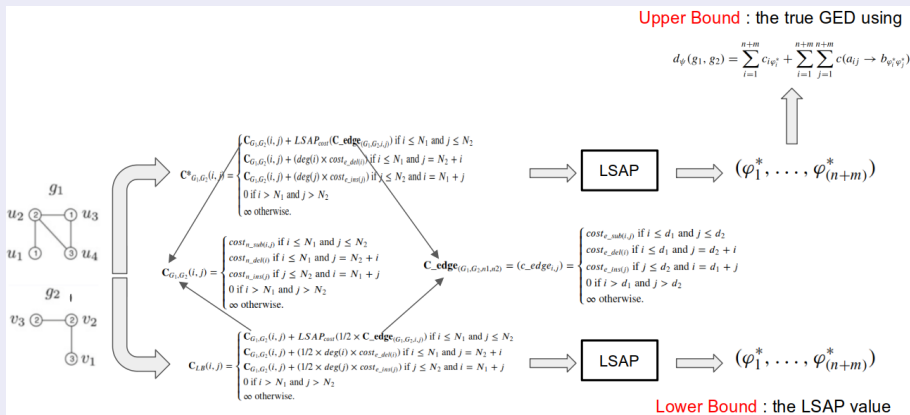
- **Lower bound**
- Idea : under-evaluate the edge matching cost using LSAP on edges and by introducing a $\frac{1}{2}$ factor

$$d'_\psi(g_1, g_2) = \min_{(\varphi_1^*, \dots, \varphi_{(n+m)}^*) \in \mathcal{S}_{(n+m)}} \sum_{i=1}^{n+m} c_{i\varphi_i^*} + \sum_{i=1}^{n+m} \left[\min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \sum_{k=1}^{n+m} \frac{c(a_{ik} \rightarrow b_{\varphi_i^* \varphi_k})}{2} \right]$$

- $d'_\psi(g_1, g_2)$ is not related to a valid edit path
- It can be shown that $d'_\psi(g_1, g_2)$ is a lower bound of the true GED

From GED to assignment problems

BP-GED framework (practical session)



From GED to assignment problems

BP-GED results

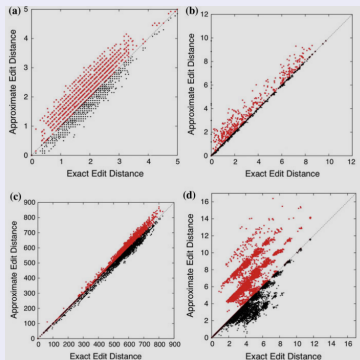


Fig. 3.2 Exact (x-axis) versus approximate (y-axis) graph edit distance computed with BP-GED resulting in d_g (red points) and d_g^{approx} (black points) on all data sets. a AIDS, b FP, c GREC, d LETTER

From GED to assignment problems

BP-GED as a seed for many methods

- BP-GED is frequently used as a starting point and combined with refinements
 - ▶ With Post-processing methods (see K. Riesen book and extra slides)
 - ▶ With improved cost matrices which better take into account the structure :
 - ★ based on node centrality (see the work of Riesen and Fisher)
 - ★ based on subgraphs (see the work of Carletti)
 - ★ based on bag of walks (see the work of Gauzere)
- BP can be used for computing bounds in tree-search methods
- A tradeoff accuracy vs complexity has to be found

Outline of the lecture

- 1 Lecturer presentation
- 2 Metrics on graphs
- 3 Exact Graph Matching based similarities
- 4 Graph Edit Distance
 - Error-Tolerant graph Matching
 - Definitions and notations
 - Algorithms for exact GED computation
 - Tree-search algorithms
 - Mathematical programming approaches
 - Algorithms for approximate GED computation
 - LSAP approaches
 - QAP approaches
- 5 The Hungarian Method
- 6 Extra-slides : BP-GED refinements

QAP approaches

Basics

- BP-GED do not fully exploit the structure of the graph
- In order to go beyond these results, the search must be guided by considering the real quadratic problem.

$$d_{\lambda_{\min}} = \min_{(\varphi_1, \dots, \varphi_{(n+m)}) \in \mathcal{S}_{(n+m)}} \left[\sum_{i=1}^{n+m} c_{i\varphi_i} + \sum_{i=1}^{n+m} \sum_{j=1}^{n+m} c(a_{ij} \rightarrow b_{\varphi_i \varphi_j}) \right]$$

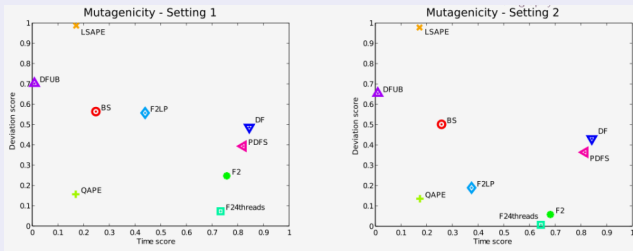
Recent propositions

- Solve the QAP problem using :
 - ▶ IPFP
 - ▶ GNCCP

See Benoit Gauzere publications for more information in this field

Conclusion

Recent results



(BS : Beam search, LSAPE (BP+RW), QAPE (IPFP), DFUB (Depth first Upper bound), DF (Depth first), F2 (ILP))

<http://authors.elsevier.com/sd/article/S0167865517303690>

Outline of the lecture

- 1 Lecturer presentation
- 2 Metrics on graphs
- 3 Exact Graph Matching based similarities
- 4 Graph Edit Distance
 - Error-Tolerant graph Matching
 - Definitions and notations
 - Algorithms for exact GED computation
 - Tree-search algorithms
 - Mathematical programming approaches
 - Algorithms for approximate GED computation
 - LSAP approaches
 - QAP approaches
- 5 The Hungarian Method
- 6 Extra-slides : BP-GED refinements

How to solve a Linear Assignment Problem

Classical approach : The Hungarian algorithm

- The Hungarian method is an iterative algorithm for solving the LSAP
- Its worst case run-time complexity is $O(n^3)$

Steps

- Step 1 : Cost Matrix reduction
 - ▶ One each row, subtract the minimum value of the line
 - ▶ On each column, subtract the minimum value of the column
- Step 2 : Make assignment and remove impossible assignment
 - ▶ From row 1 to row N if there is one uncrossed 0, frame it
 - ▶ Cross the 0 of the same column than framed 0
 - ▶ From column 1 to column N if there is one uncrossed 0, frame it
 - ▶ Cross the 0 of the same row than framed 0
- Step 3 : if there is a single framed 0 per row/column, the job is done !

How to solve a Linear Assignment Problem

Classical approach : The Hungarian algorithm

- Lets apply the algorithm with this first Matrix

$$\begin{pmatrix} 5 & 9 & 3 & 6 \\ 8 & 7 & 8 & 2 \\ 6 & 10 & 12 & 7 \\ 3 & 10 & 8 & 6 \end{pmatrix}$$

How to solve a Linear Assignment Problem

Classical approach : The Hungarian algorithm

- Lets apply the algorithm with this first Matrix

$$\begin{pmatrix} 5 & 9 & 3 & 6 \\ 8 & 7 & 8 & 2 \\ 6 & 10 & 12 & 7 \\ 3 & 10 & 8 & 6 \end{pmatrix}$$

Easy : Best assignment costs 18

$$\begin{pmatrix} 5 & 9 & \boxed{3} & 6 \\ 8 & 7 & 8 & \boxed{2} \\ 6 & \boxed{10} & 12 & 7 \\ \boxed{3} & 10 & 8 & 6 \end{pmatrix}$$

How to solve a Linear Assignment Problem

Classical approach : The Hungarian algorithm

- A second one ?

$$\begin{pmatrix} 11 & 7 & 10 & 17 & 10 \\ 13 & 21 & 7 & 11 & 13 \\ 13 & 13 & 15 & 13 & 14 \\ 18 & 10 & 13 & 16 & 14 \\ 12 & 8 & 16 & 19 & 10 \end{pmatrix}$$

How to solve a Linear Assignment Problem

Classical approach : The Hungarian algorithm

- A second one ?

$$\begin{pmatrix} 11 & 7 & 10 & 17 & 10 \\ 13 & 21 & 7 & 11 & 13 \\ 13 & 13 & 15 & 13 & 14 \\ 18 & 10 & 13 & 16 & 14 \\ 12 & 8 & 16 & 19 & 10 \end{pmatrix}$$

We have a problem : new steps are needed

How to solve a Linear Assignment Problem

Extra-Steps

- Step 4 : row and column marking
 - ▶ 4a : Mark the rows without squared 0
 - ▶ 4b : Mark the columns with crossed 0 on marked rows
 - ▶ 4c : Mark the row with squared 0 on marked columns
 - ▶ Go back to 4b if necessary
- Step 5 : line covering
 - ▶ 5a : draw a line of each unmarked row
 - ▶ 5b : draw a line of each marked column
- Step 6 : Matrix modification
 - ▶ 6a : $\theta \leftarrow$ minimum of uncovered value
 - ▶ 6b : subtract θ to each uncovered element
 - ▶ 6c : add θ for each doubly covered element
 - ▶ 6d : Other elements keep their value
- Apply Steps 2 and 3

How to solve a Linear Assignment Problem

Classical approach : The hungarian algorithm

- Lets apply these steps on the 5×5 matrix of the preceding slide (here I recall the values after reduction)

$$\begin{pmatrix} 4 & 0 & 3 & 10 & 2 \\ 6 & 14 & 0 & 4 & 5 \\ 0 & 0 & 2 & 0 & 0 \\ 8 & 0 & 3 & 6 & 3 \\ 4 & 0 & 8 & 11 & 1 \end{pmatrix}$$

Easy : Best assignment costs 51

Outline of the lecture

- 1 Lecturer presentation
- 2 Metrics on graphs
- 3 Exact Graph Matching based similarities
- 4 Graph Edit Distance
 - Error-Tolerant graph Matching
 - Definitions and notations
 - Algorithms for exact GED computation
 - Tree-search algorithms
 - Mathematical programming approaches
 - Algorithms for approximate GED computation
 - LSAP approaches
 - QAP approaches
- 5 The Hungarian Method
- 6 Extra-slides : BP-GED refinements

Improvements of BP-GED

Iterative search

- Rationale : some node assignments obtained with C^* induce a large cost when edge edit operations are computed
- Idea of Iterative search : remove these assignments using an infinite corresponding cost in C^* and re-compute a new assignment
- Iterate for q steps [by keeping bijectivity]
- Choose the best among the $q + 1$ approximations
- Algorithm (from now C states for C^* , error on i in this Alg.)

```

1: Build cost matrix  $C = (c_{ij})$  according to the input graphs  $g_1$  and  $g_2$ 
2: Compute optimal node assignment  $\psi = \{u_1 \rightarrow v_{\varphi_1}, \dots, u_{m+n} \rightarrow v_{\varphi_{m+n}}\}$  on  $C$ 
3:  $d_{best} = d_\psi(g_1, g_2)$ 
4:  $i = 0$ 
5: while  $i < q$  do
6:    $i++$ 
7:   Determine node operation  $(u_i \rightarrow v_{\varphi_i}) \in \psi$  with highest implied edge edit cost
8:   Modify  $C$  by setting  $c_{i\varphi_i} = \infty$  (prevent  $(u_i \rightarrow v_{\varphi_i})$  for future solutions)
9:   Compute optimal node assignment  $\psi'$  on modified cost matrix  $C$  and set  $\psi = \psi'$ 
10:  if  $d_{\psi'}(g_1, g_2) < d_{best}$  then
11:     $d_{best} = d_{\psi'}(g_1, g_2)$ 
12:  end if
13: end while
14: return  $d_{best}$ 

```

Improvements of BP-GED

Floating search

- Rationale : removed assignments in IS can not become valid later
- Idea of floating search : re-authorize these assignments

```

1: Build cost matrix  $C = (c_{ij})$  according to the input graphs  $g_1$  and  $g_2$ 
2: Compute optimal node assignment  $\psi = \{u_1 \rightarrow v_{\psi_1}, \dots, u_{m+n} \rightarrow v_{\psi_{m+n}}\}$  on  $C$ 
3: Initialize empty array for buffering  $(q+1)$  distance values  $d[0 \dots q]$ 
4:  $d[0] = d_\psi(g_1, g_2)$ 
5: Initialize empty list for buffering prevented edit operations  $f = \{\}$ 
6:  $i = 1$ 
7: while  $i \leq q$  do
8:   Determine node operation  $(u_k \rightarrow v_{\psi_k}) \in \psi$  with highest implied edge edit cost
9:   Modify  $C$  by setting  $c_{k\psi_k} = \infty$  (prevent  $(u_k \rightarrow v_{\psi_k})$  for future solutions)
10:  Compute optimal node assignment  $\psi'$  on modified cost matrix  $C$ 
11:   $d[i] = d_{\psi'}(g_1, g_2)$  (buffer approximation value with  $i$  preventions)
12:   $f = f \cup \{(k, \psi_k)\}$  (buffer the indices of the prevented node operation)
13:   $removed = true; j=0; \xi_1 = (-), \xi_2 = (-)$ 
14:  while  $removed$  do
15:     $removed = false; d_r = \infty; j++$ 
16:    for all indices pairs  $(k, \psi_k) \in f$  do
17:      Modify  $C$  by resetting  $c_{k\psi_k}$  to the original cost entry
18:      Compute optimal node assignment  $\psi'$  on modified cost matrix
19:      if  $d_{\psi'}(g_1, g_2) < d_r$  then
20:         $d_r = d_{\psi'}$ 
21:         $\xi_1 = (k), \xi_2 = (\psi_k)$ 
22:      end if
23:      Modify  $C$  by setting  $c_{k\psi_k} = \infty$ 
24:    end for
25:    if  $d_r < d[i-j]$  then
26:       $d[i-j] = d_r$ 
27:       $f = f - \{(\xi_1, \xi_2)\}$ 
28:       $removed = true;$ 
29:      Modify  $C$  by resetting  $c_{\xi_1\xi_2}$  to the original cost entry
30:       $\psi = \text{optimal node assignment on } C$ 
31:    else
32:       $j--$ 
33:    end if
34:  end while
35:   $i = i - j$ 
36: end while

```

Improvements of BP-GED

Genetic search

- Idea of genetic search : introduce a part of randomness

```

1: Build cost matrix  $C = (c_{ij})$  according to the input graphs  $g_1$  and  $g_2$ 
2: Compute optimal node assignment  $\psi = \{u_1 \rightarrow v_{\varphi_1}, \dots, u_{m+n} \rightarrow v_{\varphi_{m+n}}\}$  on  $C$ 
3:  $d_{best} = d_{\psi}(g_1, g_2)$ 
4: for  $i = 1, \dots, s$  do
5:   build  $P(0) = \{\psi, \psi_1^{(0)}, \dots, \psi_{N-1}^{(0)}\}$  based on  $\psi$  using mutation probability  $p$ 
6:    $t = 0; l = 0$ 
7:   while  $t - l < \delta$  do
8:     select a subset  $E \subseteq P(t)$  of parents ( $|E| = f \cdot N$ )
9:     build a new population  $P(t+1) = E \cup \{\psi_1^{(t+1)}, \dots, \psi_{N-|E|}^{(t+1)}\}$  from  $E$ 
10:     $d = \min_{i=1, \dots, N} \{d_{\psi_i^{(t+1)}}(g_1, g_2)\}$ 
11:     $t = t + 1$ 
12:    if  $d < d_{best}$  then
13:       $d_{best} = d; l = t$ 
14:    end if
15:  end while
16: end for
17: return  $d_{best}$ 

```

- s repetitions because GA are stochastic
- Variations : infinite values in C with a mutation probability p
- Elitism : a proportion f of the better parents is kept
- Child generation : $C_{child} = (\max(C_1, C_2))$

Improvements of BP-GED

Greedy search

- Idea : swaps instead of infinite costs $\rightarrow$ no new LSAP computation

```

1: Build cost matrix  $C = (c_{ij})$  according to the input graphs  $g_1$  and  $g_2$ 
2: Compute optimal node assignment  $\psi = \{u_1 \rightarrow v_{\varphi_1}, \dots, u_{m+n} \rightarrow v_{\varphi_{m+n}}\}$  on  $C$ 
3:  $d_{best} = d_{\psi}(g_1, g_2)$ 
4:  $swapped = true$ 
5: while  $swapped$  do
6:    $swapped = false$ 
7:   for  $i = 1, \dots, (m + n - 1)$  do
8:     for  $j = i + 1, \dots, (m + n)$  do
9:        $cost_{orig} = c_{i\varphi_i} + c_{j\varphi_j}$ 
10:       $cost_{swap} = c_{i\varphi_j} + c_{j\varphi_i}$ 
11:      if  $|cost_{orig} - cost_{swap}| \leq \theta \cdot cost_{orig}$  then
12:         $\psi' = \psi \setminus \{u_i \rightarrow v_{\varphi_i}, u_j \rightarrow v_{\varphi_j}\} \cup \{u_i \rightarrow v_{\varphi_j}, u_j \rightarrow v_{\varphi_i}\}$ 
13:        Derive approximate edit distance  $d_{\psi'}(g_1, g_2)$ 
14:        if  $d_{\psi'}(g_1, g_2) < d_{best}$  then
15:           $d_{best} = d_{\psi'}(g_1, g_2)$ 
16:           $best\_swap = \{i, \varphi_j, j, \varphi_i\}$ 
17:           $swapped = true$ 
18:        end if
19:      end if
20:    end for
21:  end for
22:  if  $swapped$  then
23:    update  $\psi$  according to  $best\_swap$ 
24:  end if
25: end while
26: return  $d_{best}$ 

```

- Swaps with comparable costs are considered
- Best swap is kept at each main iteration, while swaps are benefic

Improvements of BP-GED

Genetic search with Swaps

- Idea : the same as Genetic Search but with swaps
- Steps 5 and 9 are modified using the strategy of Greedy Search
- A single swap is applied at each iteration

```

1: Build cost matrix  $C = (c_{ij})$  according to the input graphs  $g_1$  and  $g_2$ 
2: Compute optimal node assignment  $\psi = \{u_1 \rightarrow v_{\varphi_1}, \dots, u_{m+n} \rightarrow v_{\varphi_{m+n}}\}$  on  $C$ 
3:  $d_{best} = d_\psi(g_1, g_2)$ 
4: for  $i = 1, \dots, s$  do
5:   build  $P(0) = \{\psi, \psi_1^{(0)}, \dots, \psi_{N-1}^{(0)}\}$  based on  $\psi$  using mutation probability  $p$ 
6:    $t = 0; l = 0$ 
7:   while  $t - l < \delta$  do
8:     select a subset  $E \subseteq P(t)$  of parents ( $|E| = f \cdot N$ )
9:     build a new population  $P(t+1) = E \cup \{\psi_1^{(t+1)}, \dots, \psi_{N-|E|}^{(t+1)}\}$  from  $E$ 
10:     $d = \min_{i=1, \dots, N} \{d_{\psi_i^{(t+1)}}(g_1, g_2)\}$ 
11:     $t = t + 1$ 
12:    if  $d < d_{best}$  then
13:       $d_{best} = d; l = t$ 
14:    end if
15:  end while
16: end for
17: return  $d_{best}$ 

```

```

1: Initialize  $P(0) = \{\psi\}$ 
2: while  $P(0) < N$  do
3:   for  $i = 1, \dots, (m+n-1)$  do
4:     for  $j = i+1, \dots, (m+n)$  do
5:        $cost_{orig} = c_{i\varphi_i} + c_{j\varphi_j}$ 
6:        $cost_{swap} = c_{i\varphi_j} + c_{j\varphi_i}$ 
7:       if  $|cost_{orig} - cost_{swap}| \leq \theta \cdot cost_{orig}$  then
8:         Generate random number  $r \in [0, 1]$ 
9:         if  $r < p$  then
10:           $\psi' = \psi \setminus \{u_i \rightarrow v_{\varphi_i}, u_j \rightarrow v_{\varphi_j}\} \cup \{u_i \rightarrow v_{\varphi_j}, u_j \rightarrow v_{\varphi_i}\}$ 
11:           $P(0) = P(0) \cup \{\psi'\}$ 
12:           $i = (m+n); j = (m+n+1)$ 
13:        end if
14:      end if
15:    end for
16:  end for
17: end while
18: return  $P(0)$ 

```

Improvements of BP-GED

Beam Search

- Idea : selecting swaps in a tree search
- Each node = {an assignment, its level, and the corresponding GED}
- The *open* list is sorted according to the level and the GED (asc.)
- At level q , the edit operation $(u_q \rightarrow v_{\varphi_q})$ is swapped

```

1: Build cost matrix  $C = (c_{ij})$  according to the input graphs  $g_1$  and  $g_2$ 
2: Compute optimal node assignment  $\psi = \{u_1 \rightarrow v_{\varphi_1}, \dots, u_{m+n} \rightarrow v_{\varphi_{m+n}}\}$  on  $C$ 
3:  $d_{best} = d_\psi(g_1, g_2)$ 
4: Initialize  $open = \{(\psi, 0, d_\psi(g_1, g_2))\}$ 
5: while  $open$  is not empty do
6:   Remove first tree node in  $open$ :  $(\psi, q, d_\psi(g_1, g_2))$ 
7:   for  $j = (q + 1), \dots, (m + n)$  do
8:      $\psi' = \psi \setminus \{u_{q+1} \rightarrow v_{\varphi_{q+1}}, u_j \rightarrow v_{\varphi_j}\} \cup \{u_{q+1} \rightarrow v_{\varphi_j}, u_j \rightarrow v_{\varphi_{q+1}}\}$ 
9:     Derive approximate edit distance  $d_{\psi'}(g_1, g_2)$ 
10:     $open = open \cup \{(\psi', q + 1, d_{\psi'}(g_1, g_2))\}$ 
11:    if  $d_{\psi'}(g_1, g_2) < d_{best}$  then
12:       $d_{best} = d_{\psi'}(g_1, g_2)$ 
13:    end if
14:  end for
15:  while size of  $open > b$  do
16:    Remove tree node with highest approximation value  $d_\psi$  from  $open$ 
17:  end while
18: end while
19: return  $d_{best}$ 

```

- lines 15-17 prevent a complete exploration of the tree by bounding the size of *open*

Improvements of BP-GED

Improved Beam Search

- Idea : in BS, nodes are processed in the order defined by the graph g_1
- Refinement : SBP-Beam : the original assignment is re-ordered
- 6 different criteria are proposed in [Ferrer, Serratos, Riesen PRL]

```

1:  $d_\psi(g_1, g_2) = BP - GED(g_1, g_2)$ 
2:  $\psi' = Sort(\psi)$ 
3:  $d_{SortedBeam}(g_1, g_2) = BP-Beam(g_1, g_2, d_\psi, \psi', b)$ 
4: return  $d_{SortedBeam}(g_1, g_2)$ 

```